

**SVEUČILIŠTE U SPLITU
FAKULTET ELEKTROTEHNIKE, STROJARSTVA I
BRODOGRADNJE**

ZAVRŠNI RAD

**SIMULACIJA DIGITALNIH SKLOPOVA
KORIŠTENJEM React.js**

Dominko Milić

Split, svibanj 2025.



Sveučilišni prijediplomski studij: **Računarstvo**

Smjer/Usmjerenje: /

Oznaka programa: 120

Akadska godina: 2024./2025.

Ime i prezime: **Dominko Milić**

JMBAG: 0023157320

ZADATAK ZAVRŠNOG RADA

Naslov: **SIMULACIJA DIGITALNIH SKLOPOVA KORIŠTENJEM REACT.JS**

Zadatak: Korištenjem React.js-a izraditi web aplikaciju koja osigurava uvid u ponašanje različitih digitalnih sklopova korištenjem grafičkih komponenti korisničkog sučelja. U radu definirati karakteristične pojmove koji su važni za razumijevanje funkcioniranja logičkih sklopova i dati pregled postojećih simulacijskih rješenja. Predstaviti tehnologiju korištenu za izradu aplikacije, s naglaskom na karakteristične značajke koje su važne za performanse i mogućnost stvaranja rješenja. Opisati funkcionalnosti aplikacije i način na koji su dostupne putem grafičkih komponenti sučelja. Istaknuti posebnosti aplikacije u smislu mogućnosti spremanja i učitavanja datoteka unaprijed izgrađenih logičkih sklopova.

Datum obrane: 27.06.2025.

Mentor:

prof. dr. sc. Mirjana Bonković

IZJAVA

Ovom izjavom potvrđujem da sam završni rad s naslovom SIMULACIJA DIGITALNIH SKLOPOVA KORIŠTENJEM React.js-a pod mentorstvom prof. dr. sc. MIRJANE BONKOVIĆ pisao samostalno, primijenivši znanja i vještine stečene tijekom studiranja na Fakultetu elektrotehnike, strojarstva i brodogradnje, kao i metodologiju znanstveno-istraživačkog rada, te uz korištenje literature koja je navedena u radu. Spoznaje, stavove, zaključke, teorije i zakonitosti drugih autora koje sam izravno ili parafrazirajući naveo u završnom radu citirao sam i povezao s korištenim bibliografskim jedinicama.

Student

A handwritten signature in blue ink, consisting of stylized, overlapping loops and strokes, representing the name Dominko Milić.

Dominko Milić

SADRŽAJ

1. UVOD	1
2. LOGIČKI SKLOPOVI	2
2.1. Osnovni elementi	2
2.2. Složeni elementi	2
2.3. Postojeća rješenja	6
3. KORIŠTENE TEHNOLOGIJE	8
3.1. Microsoft Visual Studio Code	8
3.2. React.js	14
4. APLIKACIJA	24
4.1. Grafičko sučelje	24
4.2. Struktura elemenata	27
4.3. Algoritam za izračun izlazne vrijednosti	28
4.4. Predefinirani i učitani digitalni krugovi	31
5. ZAKLJUČAK	33
LITERATURA	34
POPIS OZNAKA I KRATICA	36
SAŽETAK	37
SUMMARY	38

1. UVOD

U novije vrijeme razumijevanje i analiza rada digitalnih sklopova sve je potrebna vještina za izradu što boljih računalnih i elektroničkih sustava. Alati za simulaciju omogućuju studentima i inženjerima da bez potrebe za stvarnim spajanjem sklopova testiraju njihov rad i mogućnosti. Upravo zbog toga raste potreba za modernim, interaktivnim i jednostavnim aplikacijama koje omogućuju brzo i vizualno razumijevanje digitalnih sklopova.

Rad detaljno predstavlja sve segmente potrebne za efikasno korištenje aplikacije koji uključuju: predstavljanje rada pojedinih digitalnih sklopova i njihovih jednostavnih kombinacija, tehnologiju korištenu za razvoj aplikacije i način na koji su dijelovi aplikacije implementirani. Konačno, opisane su funkcionalnosti aplikacije i mogućnosti za pohranjivanje generiranih rezultata i kreiranih sklopova. Ukratko, aplikacija omogućuje korisniku dodavanje raznih logičkih sklopova te njihovo međusobno kombiniranje. Elementi se mogu povezivati međusobno ili na generator, to jest indikator. Tako povezani elementi uzimaju početne vrijednosti s generatora te ih nakon obrade kroz pojedine elemente prikazuju na indikatorima.

Rad je podijeljen na pet cjelina. Drugo poglavlje općenito opisuje digitalne sklopove, osnovne i složenije. U trećem poglavlju opisane su sve korištene tehnologije. Četvrto poglavlje opisuje aplikaciju i proces njene izrade. Na kraju zaključak zaokružuje cijeli rad u zadnjem istoimenom poglavlju.

2. LOGIČKI SKLOPOVI

Logički sklopovi su elementi namijenjeni izvođenju određenih logičkih funkcija. Primjenjuju se u računalima, regulacijskim krugovima, u uređajima za daljinska mjerenja i upravljanja i dr. Postoje osnovni logički sklopovi koji provode osnovne logičke operacije dok se njihovim kombiniranjem mogu ostvariti sve ostale operacije. [1]

2.1. Osnovni elementi

Osnovna logička vrata su sastavni dio svih digitalnih uređaja te su temelj za projektiranje računala i mikrokontrolera. Takvi elementi se temelje na osnovnim logičkim operacijama te obrađivanjem binarnih signala 0 i 1 daju izlaz, također 0 ili 1.

Osnovni elementi su:

I(AND) AND logički sklop na ulaz prima n argumenata te za izlaz daje 1 samo ako su svi ulazi jednaki 1

ILI(OR) OR logički sklop na ulaz prima n argumenata te na izlaz daje 1 ako je bilo koji ulaz jednak 1

NE(NOT) NOT logički sklop daje vrijednost izlaza obrnut od vrijednosti ulaza, za ulaz 0 vraća izlaz 1 i obratno

NI(NAND) NAND logički sklop je inverzni AND, za iste ulaze vraća obrnuti izlaz

NILI(NOR) NOR logički sklop je inverzni OR, za iste ulaze vraća obrnuti izlaz

ISKLJUČIVI ILI(XOR) XOR logički sklop za izlaz vraća 1 ako su ulazi različiti, tj. 1 i 0

ISKLJUČIVI NILI (XNOR) XNOR logički sklop je inverzni XOR, na izlaz vraća 1 ako su svi ulazi jednaki

2.2. Složeni elementi

Kombiniranjem jednostavnih elemenata u određene cjeline, dobivamo složene logičke sklopove koji se dijele na dvije osnovne kategorije: kombinacijski i sekvencijalni sklopovi. Ti elementi se koriste u raznim komponentama od računalne arhitekture za registre i memoriju,

preko multipleksiranja signala u telekomunikacijama do kontrole upravljačkih sustava pomoću automata i sekvencera te mnogih drugih područja.

Elementi koji se mogu koristiti u kreiranoj aplikaciji su multiplekser, demultiplekser, D i JK bistabili te enkoder prioriteta, ali osim ovih integriranih elemenata, aplikacija omogućava realizaciju ostalih kompleksnih elemenata kombiniranjem osnovnih logičkih vrata.

MUX (multiplekser) je element koji od više ulaznih signala, na temelju signala na upravljačkim ulazima, na izlaz šalje jedan izlazni signal. Koristi se kada se šalje više signala preko jednog kanala [5]. U ovoj web aplikaciji se mogu pronaći MUX-i s dva, četiri te osam adresnih ulaza, koji odgovaraju broju od jednog(model 74xx157), dva(model 74xx153) i tri(model 74xx151) upravljačka ulaza. Svaki od zadanih MUX-a ima dva izlaza, stvarni izlaz te inverzni izlaz.

Za razliku od MUX-a, DEMUX(demultiplekser) je sklop koji od jednog ulaznog signala, ovisno o signalima na upravljačkim ulazima, daje više izlaza. U stvarnosti se koristi za usmjeravanje podataka na više odredišta [6]. U aplikaciji se mogu koristiti integrirani DEMUX-i s četiri adresna izlaza, tj. s dva upravljačka ulaza(model 74xx139) te DEMUX s osam adresnih izlaza i tri upravljačka ulaza(model 74xx138) koji određuju izlaze.

D(delay) bistabil korišten u ovoj aplikaciji je model 74xx74 koji ima asinkrone RS ulaze i s taktnim ulazom osjetljivim na ulazni brid taktnog signala. To je sekvencijalni sklop koji pamti jedan bit podatka u kojem se pri svakom taktu ulaz D kopira na izlaz Q. Ovaj bistabil se koristi za memoriju, registriranje podataka i sinkronizaciju. Jednadžba koja se koristi za računanje izlaza na D bistabilu je $Q(n+1) = D(n)$ gdje $Q(n+1)$ predstavlja izlaz za idući ciklus, a $D(n)$ je ulaz u trenutnom ciklusu. n predstavlja trenutni ciklus. Karakteristična tablica za D bistabil je prikazana na slici 2-1.

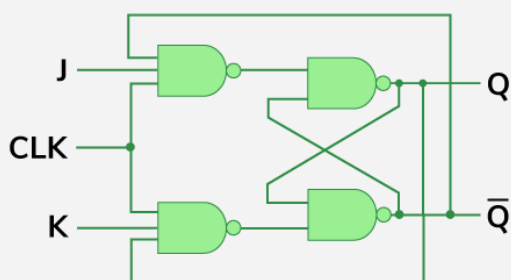
D	Q(Current)	Q(n+1) (Next)
0	0	0
0	1	0
1	0	1
1	1	1

slika 2-1 Karakteristična tablica D bistabila [7]

Još jedan bistabil koji je integriran u ovaj rad je JK(Jack Kilby) bistabil modela 74xx112 koji sadrži asinkrone RS ulaze i takti ulaz osjetljiv na silazni brid taktnog signala. To je univerzalni sekvencijski sklop koji se koristi za izradu brojača, registara te za sinkronizaciju. Rad ovog bistabila prikazan je na slici 2-2. Ova tablica se može podijeliti na četiri slučaja:

Za slučaj $J=0$ i $K=0$ $Q(n+1)$, tj. iduće stanje, ostaje jednako trenutnom stanju $Q(n)$. Za „reset“ slučaj, $J=0$ i $K=1$, iduće stanje se vraća na 0 bez obzira o trenutnom stanju. „Set“ slučaj, $J=1$ i $K=0$, iduće stanje postavlja na 1, tj. $Q(n+1) = 1$ ne oviseći pri tom o trenutnom stanju. „Toggle“ slučaj u kojem su $J=1$ i $K=1$ čini da se iduće stanje promijeni, ako je $Q(n+1)$ bilo 0 sad će biti 1 i obratno.

JK Flip-Flop

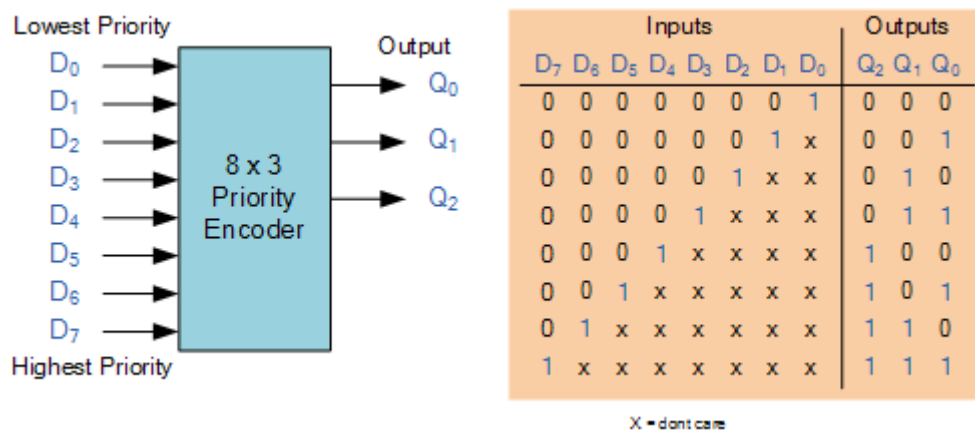


Truth Table

J	K	Q_N	Q_{N+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

slika 2-2 JK bistabil i njegova karakteristična tablica [8]

Posljednji složeni logički sklop koji je integriran u aplikaciju rada je enkoder prioriteta modela 74xx148 koji ima osam adresnih ulaza te tri adresna izlaza te ima mogućnost serijskog povezivanja na druge enkodere ako je potrebno više od osam ulaza. To je sklop koji u prirodnom binarnom obliku generira redni broj aktivnog ulaza. Koristan je kada se u sustavima treba odrediti tko ima prednost, npr. pri prepoznavanju prekida u procesorima. Na slici 2-3 je prikazan rad upravo ovog enkodera prioriteta te je vidljivo da su izlazi ovisni samo o najznačajnijem ulaznom bitu dok su ostali nevažni (označeni oznakom x). Kombinacija izlaza je praktički decimalni broj koji simbolizira broj ulaza na kojem se nalazi najznačajniji bit koji ima vrijednost 1.



slika 2-3 Simbol i karakteristična tablica enkodera prioriteta s osam adresnih ulaza [9]

2.3. Postojeća rješenja

Kako se znanost o digitalnim sklopovima u današnje vrijeme sve više razvija, sve veća je potreba i za simulatorima koji se bave takvim sklopovima. Neki od postojećih alata koji se koriste za simuliranje logičkih sklopova su Logisim, Digital, Logicly te Tinkercad.

- Logisim

Logisim je besplatni open-source program koji radi na bilo kojem uređaju koji podržava Java 5 ili novije verzije. Cilj ove aplikacije je pomoć studentima pri dizajniranju te simuliranju logičkih sklopova. Sučelje aplikacije je intuitivno što korisnicima olakšava uporabu aplikacije. Kreirani logički sklopovi mogu biti spremljeni u obliku slike ili GIF-a. [2]

- Digital

Digital je još jedan besplatni open-source program koji je također namijenjen za edukacijske svrhe. Preuzimanje ovog programa radi se preko GitHuba te se pokreće standardnom naredbom za pokretanje java programa preko terminala. Dodatna značajka ovog simulatora jest što prikazuje i razradu karnaugove tablice te tablice istine. [3]

- Logicly

Logicly je aplikacija koja intuitivnim korisničkim sučeljem na jednostavan način omogućava korisnicima da dodaju, brišu, spajaju i kreiraju nove logičke sklopove, od jednostavnijih do složenijih. Dostupna je besplatna probna verzija, dok se puna verzija aplikacije plaća. Podržava Windows i macOS okruženje. [4]

- Tinkercad

Tinkercad je besplatna web aplikacija koja omogućava 3D dizajniranje, elektroniku i kodiranje. Za razliku od prethodnih aplikacija, Thinkercad omogućava korištenje raznih elemenata i izvan logičkih sklopova kao što su LED, baterija, otpornici i mnogi drugi.

3. KORIŠTENE TEHNOLOGIJE

U svrhu izrade aplikacije korištene su sljedeće tehnologije: Microsoft Visual Studio Code i React.js te će kroz ovo poglavlje biti detaljnije opisane.

3.1. Microsoft Visual Studio Code

VS (Visual Studio) Code je besplatni, open-source uređivač koda kojeg je tvrtka Microsoft prvi put objavila u travnju 2015. godine. Od tada je postao jedan od najkorištenijih razvojnih okruženja zbog svestranosti, performanse i bogatog ekosustava proširenja za mnoge jezike.

VS Code pruža sveobuhvatne alate i funkcionalnosti koje olakšavaju razvoj softvera. To uključuje bogat skup editora za kodiranje, podršku za mnoge jezike programiranja (uključujući C++, C#, JavaScript, Python i mnoge druge), debugger za otklanjanje grešaka, upravljanje verzijama, dizajnerske alate za izradu korisničkog sučelja i mnoge druge mogućnosti.

VS Code uključuje uređivač koda koji podržava IntelliSense (komponentu za dovršavanje koda) kao i refaktoriranje koda. Sadrži integrirani debugger koji radi na razini izvornog koda te integrirani terminal za izvršavanje shell naredbi direktno u editoru. Kroz Extensions Marketplace omogućava ugradnju raznih proširenja kao što su teme, programski jezici, debuggeri i ostalo. Još jedna od velikih značajki je integrirana podrška za Git koja omogućava kontrolu verzija, povijest commita te izrada novih grana izravno u editoru. Možda i najvažnija stavka velike popularnosti VS Coda među programerima je to što je ovaj editor dostupan za Windows, macOS te Linux OS (operacijskim sustavima).

VS Code podržava različite programske jezike i omogućuje da uređivač koda i debugger podržavaju (u različitim mjerama) gotovo svaki programski jezik, pod uvjetom da postoji jezična usluga specifična za taj jezik. Neki od najvažnijih jezika koje VS Code podržava su: JavaScript, TypeScript, Python, C#, C++, HTML, Java, JSON, PHP, Markdown, Powershell, YAML, ali i mnogi drugi koji su podržavani proširenjima.[1]

Budući da VS Code podržava kodiranje preko instalacije na lokalnom uređaju, remote povezivanjem na cloud ili Git repozitorijem te na webu preko preglednika, jedan od slogana VS Coda je: „Code wherever you're most productive, whether you're connected to the cloud, a remote repository, or in the browser with VS Code for the Web (vscode.dev).“ [11] koji promovira editor koji više nije vezan za jedno mjesto i jedan uređaj.

3.1.1. Povijest

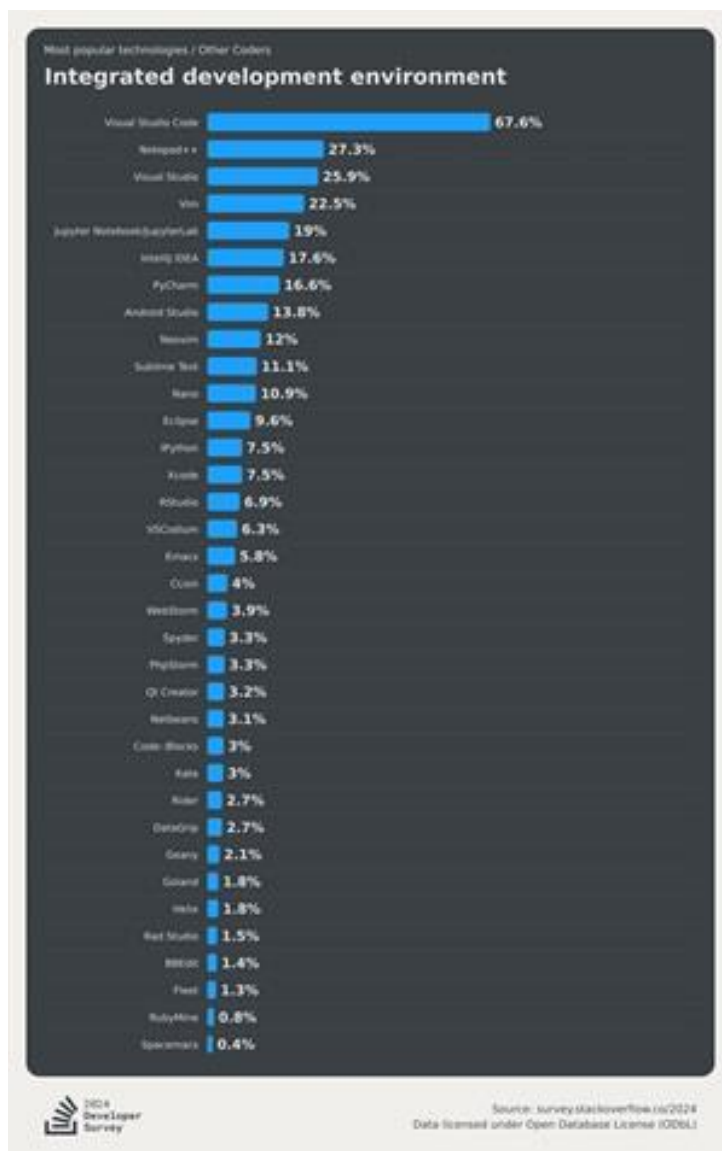
U svijetu gdje svakim danom raste potreba za novim programerskim rješenjima, nastanak novih tehnologija za pomoć pri programiranju je bilo neizbježno. VS Code je jedan od primjera takvih tehnologija koje programerima uvelike olakšavaju produktivnost i razvoj novih tehnoloških rješenja.

2011. godine Microsoftu se pridružuje Eric Gamma koji inicira razvoj editora „Monaco“ koji nakon dvije godina razvoja postaje prva službena verzija VS Codea koja je objavljena 29. travnja 2015. godine. Cilj razvoja takvog code editora je bio pokušaj Microsofta da se suprotstavi drugim tvrtkama koje su razvijale svoje OS i alate za razvoj programa. Na veliko iznenađenje mnogih korisnika, VS Code je bio besplatan, open-source i dostupan za Windows, macOS te Linux OS što je za Microsoft značilo velik preokret u dotadašnjem poslovanju.

2016. godine Microsoft, pod MIT licencom na GitHub-u, objavljuje izvorni kod VS Codea koji otvara mogućnost da svi developeri doprinose daljnjem razvijanju i napredovanju editora.

Kroz iduće godine popularnost VS Codea eksponencijalno raste zbog njegovih redovitih ažuriranja, ali i zbog sve opsežnijeg tržišta proširenja koji su omogućili korištenje više programskih jezika i veću mogućnost prilagođavanja samog editora za svakog developera.

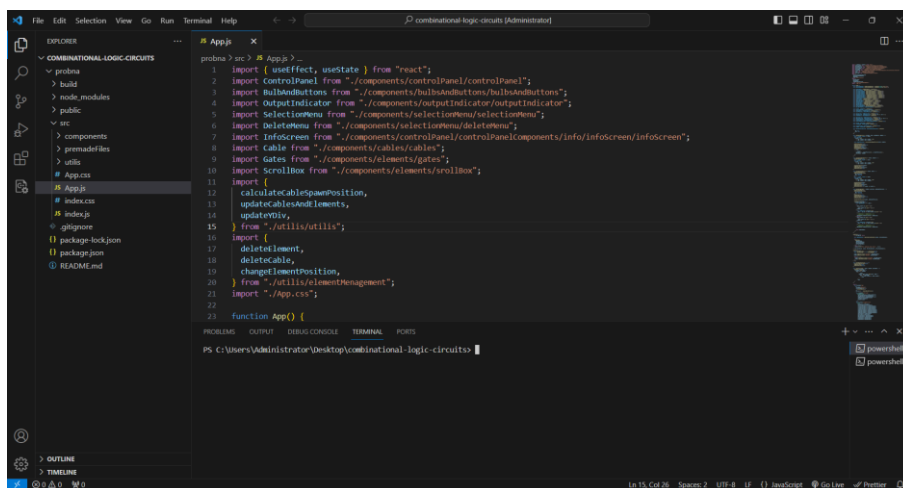
Od 2021. do 2024. godine Stack Overflow Developer Survey daje podatke o moći VS Codea naspram ostalih sličnih editora, gdje se saznaje da je u 2024. godini VS Code korišten dvostruko više od iduće alternative (Notepad++) što je vidljivo iz rezultata ankete na slici 3-1.



slika 3-1 Rezultati ankete o IDE iz 2024. [12]

Novi veliki dodatak, iz svibnja 2025. godine, za VS Code je GitHub Copilot, AI asistent koji je predviđen kao pomoć developerima za veću produktivnost i učinkovitost u radu.

Kroz godine razvoja korisničko sučelje VS Codea dobilo je moderan, intuitivan i jednostavan izgled sučelja koje pruža mnoge opcije za osobno uređivanje prikaza editora. Sadrži integriran Git i panel koji prikazuje terminal, probleme, output ili debug konzolu.



slika 3-2 Prikaz VS Code okruženja

3.1.2. Značajke

- Razvojne tehnologije

VS Code je originalno napisan u kombinaciji JavaScript i TypeScript programskih jezika, ali se ubrzo nakon objavljivanja u cijelosti prebacio u TypeScript. Razlog prebacivanja je taj što je JavaScript dinamički jezik koji je na velikim projektima podložniji greškama i bugovima pri izvođenju, dok je TypeScript statički jezik u kojem se greške mogu otkriti i za vrijeme razvoja koda. Osim toga, zbog održavanja i konzistentnosti programiranja u jednom jeziku uzet je TypeScript koji je u jednu ruku dodatno pogodovao Microsoftu, koji je razvio taj programski jezik, jer su na taj način promovirali njegovu moć te pokazali uvjerenost u rad njihovog jezika.

Koristeći Electron kao aplikacijski framework omogućeno je korištenje VS Codea na različitim OS te daje podršku za ARM-bazirane čipove. Electron je dodan na Chromium i Node.js koji su omogućili korištenje standardnih web API-ja koji pružaju stabilnost i dozvoljavaju mrežnu komponentu za GitHub s gotovo identičnom kodnom bazom. [13]

- Uređivač koda (Code Editor)

Uređivač koda podržava isticanje sintakse i dovršavanje koda pomoću IntelliSense-a za varijable, funkcije, metode i petlje. IntelliSense je podržan za uključene jezike, kao i za XML, Cascading Style Sheets i JavaScript prilikom razvoja web stranica i web aplikacija. Prijedlozi automatskog dovršavanja pojavljuju se u prozoru uređivača koda u blizini kursora za uređivanje.

Uređivač koda također podržava postavljanje oznaka u kodu za brzu navigaciju. Ostala sredstva za navigaciju uključuju skupljanje blokova koda i inkrementalno pretraživanje, osim uobičajenog pretraživanja teksta i pretraživanja s regularnim izrazima. Uređivač koda također uključuje višestruki međuspremnik i popis zadataka. Uređivač koda podržava isječke koda, koji su spremjeni predlošci za ponavljajući kod i mogu se umetnuti u kod te prilagoditi projektu na kojem se radi. Ugrađen je i alat za upravljanje isječcima koda. Ti se alati prikazuju kao plutajući prozori koji se mogu postaviti da se automatski sakriju kada se ne koriste ili da se pričvrste sa strane zaslona.

- Debugger

VS Code koristi integrirani debugger koji developerima omogućuje izravno rješavanje bugova unutar editora što olakšava sam proces debugiranja jer se izbjegava mijenjanje između različitih alata.

Debugger VS Codea omogućuje postavljanje prekidača (koji omogućuju privremeno zaustavljanje izvršavanja na određenoj poziciji) i praćenje (koje prati vrijednosti varijabli kako izvršavanje napreduje). Prekidači mogu biti uvjetni, što znači da se aktiviraju kada je zadovoljen određeni uvjet. Kod se može korak po korak izvršavati, tj. izvršava se jedna linija (izvornog koda) odjednom. Može se zakoračiti u funkcije kako bi se debugiralo unutar njih ili zakoračiti preko njih, tj. izvršavanje tijela funkcije nije dostupno za ručnu inspekciju. Debugger podržava Uredi i Nastavi (Edit and Continue), što znači da se kod može uređivati dok se debuggira.

- IntelliSense

Ova značajka omogućava automatsko dovršavanje koda, koje developerima olakšava na način da je kod pisan s manje grešaka i većom učinkovitošću. Glavne značajke VS Code IntelliSensea je dovršavanje koda, informacije o parametrima te kratke informacije. Osim službenog naziva često se koriste i nazivi: „code completion“, „content assist“ i „code hinting“.

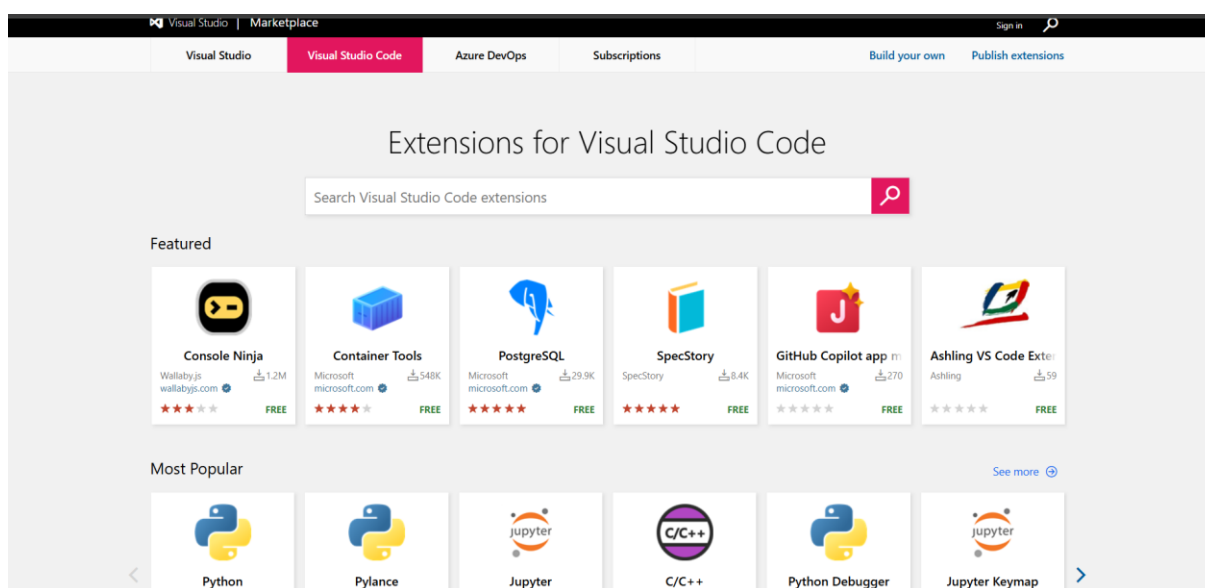
- Git Integration

Integracijom Git-a u editor developeri mogu pregledavati verzije, pratiti promjene, commitati promjene, stvarati nove grane, rješavati merge konflikte te kolaborirati na raznim projektima bez napuštanja VS Codea. To je nova pogodnost koja je dodatno olakšala i unaprijedila

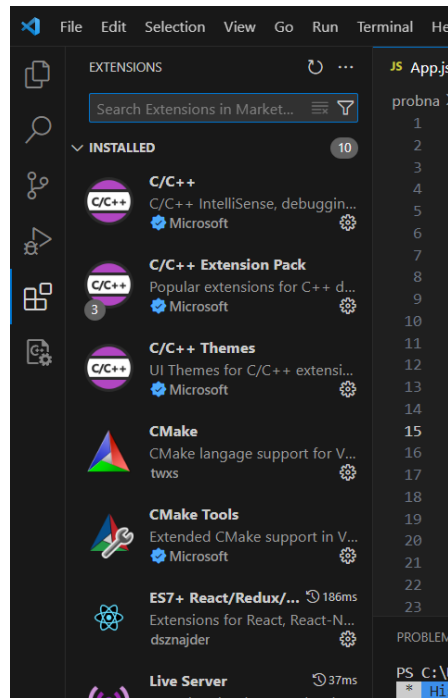
programersko iskustvo. [15] Osim navedenih stvari VS Code nudi razna proširenja koja vizualno olakšavaju i pomažu pri prezentaciji Git promjena unutar projekta, jedno od pet najpopularnijih proširenja je GitLens koja pruža detaljan pregled commit povijesti i kolaboracija.

- Tržište proširenjima

S preko 60,000 proširenja developerima se nudi mnogo opcija, što po pitanju uređivanja okruženja, što po pitanju dodavanja raznih programskih jezika i frameworka. Proširenja je moguće instalirati preko weba (slika 3-3) ili izravno u editoru (slika 3-4).



slika 3-3 Web trgovina s VS Code proširenjima [15]



slika 3-4 Integrirana trgovina proširenja

3.2. React.js

React.js također poznat i kao React ili ReactJS je jedna od najpopularnijih, besplatnih, open-source front-end JavaScript biblioteka koja se koristi za izradu korisničkih sučelja web aplikacija. Izrada takvih sučelja se temelji na React komponentama (slika 3-5).

React je koristan za izradu single-page, mobilnih ili serverskih aplikacija u kombinaciji s frameworkcima poput Next.js ili Remix. Ključna prednost Reacta naspram ostalih rješenja za izradu web aplikacija je ta što se na stranici ponovno učitaju samo podaci koji se mijenjaju dok se izbjegava nepotrebno ponovno učitavanje nepromijenjenih DOM (Document Object Model)

elemenata.

```
Video.js

function Video({ video }) {
  return (
    <div>
      <Thumbnail video={video} />
      <a href={video.url}>
        <h3>{video.title}</h3>
        <p>{video.description}</p>
      </a>
      <LikeButton video={video} />
    </div>
  );
}
```

slika 3-5 Kombiniranjem komponenti se stvara web stranica [16]

React koristi JSX (JavaScript Extensible Markup Language), proširenje za sintaksu JavaScripta koji korisnicima omogućuje pisanje koda koji podsjeća na kod HTML (Hypertext Markup Language) (slika 3-6). Koristi se radi lakšeg stvaranja i upravljanja elementima unutar JavaScript datoteka.

```
const element = <h1>Hello, world!</h1>;
```

slika 3-6 JSX sintaksa [17]

U Reactu korisničko sučelje se dijeli na manje dijelove koji se kombiniraju i na taj način stvaraju aplikaciju. Jednom kreirane komponente korisničkog sučelja mogu se ponovno koristiti u drugim React projektima što developerima omogućava lakšu, bržu i učinkovitiju izradu web stranica. Također, neovisna izrada komponenti olakšava većim timovima developera da neometano izrađuju zasebne komponente koje se na kraju jednostavno međusobno ukomponiraju u konačnu aplikaciju te olakšava održavanje i skalabilnost koda.

Funkcijske komponente se koriste za najjednostavniji način zapisa komponenti u JavaScript obliku prikazano na slici 3-7. Mogu primiti parametre, a vraćaju JSX. Također jedna od osnovnih značajki Reacta su i React Hooks koje developeri koriste za praćenje promjena u pojedinim varijablama. Neke od React Hookova su useState, useEffect, useReducer...

```
function Welcome(props) {  
  return <h1>Hello, {props.name}!</h1>;  
}
```

slika 3-7 Funkcijska komponenta

3.2.1. Povijest

Prvo razvijanje Reacta, tada pod imenom FaxJS, krenulo je 2010. godine kada je Facebook, današnji Meta, razvio prototip koji je kasnije evoluirao u React. Nakon stvaranja Instagrama, u kojeg se ulagalo sve više novih Facebookovih tehnologija, u 2012. godini stvara se potreba da React postane open-source što se i dogodilo godinu dana kasnije u svibnju 2013. godine. Kroz iduće godine Reactova popularnost je brzo rasla što je proguralo React u glavni tok za web razvoj, osobit doprinos u širenju Reacta donijeli su Netflix i Airbnb koji su se koristili tom tehnologijom.

2015. godine u React se dodaje React Native koji omogućava izradu mobilnih aplikacija koristeći JavaScript i React komponente. Ta novost privukla je mnoge mobilne developere koji su sada mogli raditi aplikacije za iOS (iPhone OS) te Android. [18]

Refakturiranje koda koji se koristio za generiranje React komponenti, React Fiber, 2017. godine poboljšalo je responzivnost i renderabilnost korisničkih sučelja React aplikacija. React Fiber radi na principu novog algoritma koji dijeli renderiranje na komade koje raspršuje preko više slika. Ovaj način renderiranja se naziva inkrementalno renderiranje i omogućava Reactu da se prvo ažuriraju bitniji dijelovi poput korisničkih interakcija dok dijelovi manjeg prioriteta čekaju. [19]

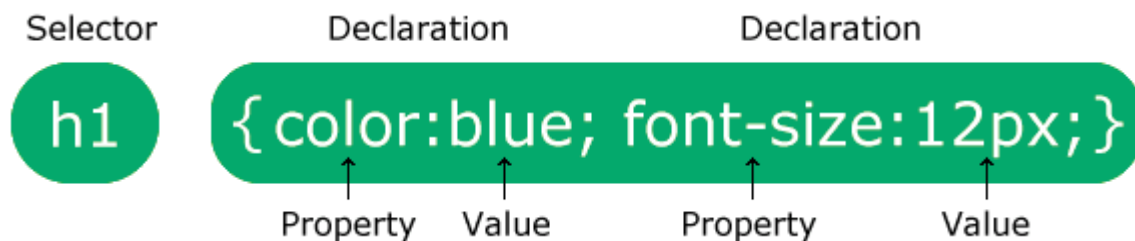
Posljednja velika novost uvedena u React 2019. godine je React Hooks koje čine kod jednostavnijim i urednijim jer se izbjegava potreba za klasama koji više ne trebaju upravljati varijablama, već to čine React Hookovi unutar funkcijskih komponenti. Ta novost je uvelike promijenila način na koje su se dotadašnje React aplikacije pisale te se popularnost Reacta ponovno povećala.

3.2.2. Značajke

- Cascading Style Sheets

CSS (Cascading Style Sheets) je jezik koji se koristi za stiliziranje web aplikacija te je njegova osnovna funkcija odvajanje sadržaja od izgleda i dizajna stranice.

CSS se piše po određenim pravilima, prvo se navodi selektor koji može biti cijeli element (p), određena klasa (.klasa) te id (#id). Nakon toga se unutar vitičastih zagrada postavlja svojstvo koje se mijenja (color, font-size, background...) i onda vrijednost na koju se svojstvo mijenja (red, 16px, center). Slika 3-8 prikazuje sintaksu pisanja CSS stila.



slika 3-8 CSS sintaksa [20]

U React.js-u CSS stilovi se mogu dodavati na više načina, a ovdje ću nabrojati neke od osnovnih. Klasični CSS je način dodavanja gdje se radi zasebna datoteka stil.css u kojoj pišu sva pravila za dizajn web stranice. U .jsx datoteku se takva datoteka ubacuje linijom „import './stil.css';“ te se sve klase, idjevi i elementi mogu dizajnirati u datoteci stil.css. Inline stilovi se pišu izravno u liniju koda gdje se neki element poziva, a primjer inline stila je „return <h1 style={{fontSize: 16px}}>Pozdrav!</h1>“; gdje će točno ovaj h1 element imati veličinu fonta 16 piksela. Još jedan od uobičajenih načina zapisa stilova u REact.js-u je CSS-in-JS koji se odnosi na definiranje stila unutar .jsx datoteke kao što je vidljivo na primjeru koda:

```
import styled from 'styled-components';

const Naslov = styled.h1`
  color: red;
  font-size: 2rem;
`;

function App() {
  return <Naslov>Pozdrav!</Naslov>;
}
```

- Virtualni DOM

VDOM (Virtualni DOM) je koncept programiranja gdje VDOM reprezentira korisničko sučelje koje se sprema u memoriju i sinkronizira sa stvarnim DOM-om pomoću biblioteka poput ReactDOM. Ovaj pristup omogućava Reactu da se DOM u potpunosti sinkronizira sa stvarnim izgledom korisničkog sučelja. Na ovaj način se uključuje automatsko kontroliranje DOM-a, rukovanje događajima i manipulacija atributima koja bi se inače morala kontrolirati ručno.

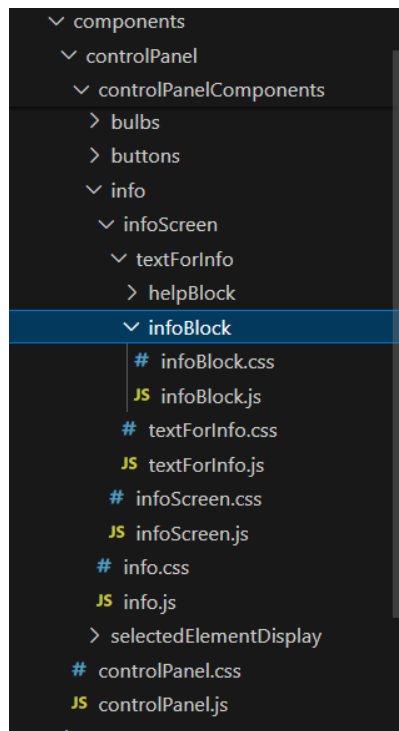
S obzirom da VDOM nije određena tehnologija ljudi ga povezuju s različitim stvarima, ali u kontekstu Reacta on se povezuje s React elementima koji čine korisničko sučelje. Budući da React svoje elemente sprema i u stablo komponenti preko React Fibera, ono se također uzima kao dio React VDOM-a. [21]

- Arhitektura bazirana na komponentama

Arhitektura bazirana na komponentama developerima savjetuje da svaki dio koda koji se može podijeliti na manje dijelove da se podijeli. Takav pristup kodiranju omogućava lakše, urednije i kvalitetnije uređivanja koda jer se promjenom jedne komponente ne ometa rad ostalih komponenti.

Jedna od prednosti ovako strukturiranog koda je ta što više developera može raditi na istom projektu, na različitim komponentama bez međusobnog ometanja napretka. Takav tip modularnog dizajna najbolje odgovara za izradu velikih aplikacija jer omogućava jednostavnu skalabilnost.

Na slici 3-9 je vidljivo korištenje arhitekture bazirane na komponentama gdje osnovna komponenta `controlPanel`, koja je uključena na glavni dio web stranice, sadrži sve jednostavnije i jednostavnije komponente koje zajedno kombinirane daju gotovi izgled korisničkog sučelja za prikaz kontrolne ploče.



slika 3-9 Prikaz arhitekture bazirane na komponentama

- JSX

JSX je proširenje JavaScript sintakse koje se koristi za opisivanje korisničkog sučelja u Reactu. JSX je uveden kako bi olakšao i optimizirao pisanje kodova na način da se odmiče od standardnog pisanja markup jezika i logike u različite datoteke te ih kombinira u jednu zajedničku. Takva datoteka je jednostavna za čitanje te osigurava izbjegavanje grešaka u aplikaciji jer developera o istima upozorava. Ono što spremanje logičkog i vizualnog dijela komponente u jednu datoteku ostvaruje je lakše održavanje koda s obzirom da se pri svakom uređenju dijela koda provjerava i drugi dio koda.

JSX se dijelom razlikuje i od HTML-a i od JavaScripta te stoga ima i svoja pravila pisanja. Funkcije koje vraćaju JSX element ne smiju vraćati više elemenata, već samo jedan te se stoga svi elementi koje funkcija vraća obavijaju jednom roditeljskom oznakom kao na slici 3-1. Ta oznaka može biti bilo koji element, ali se najčešće koristi ili `<div></div>` ili fragment `<></>`.

```

<div>
  <h1>Hedy Lamarr's Todos</h1>
  
  <ul>
    ...
  </ul>
</div>

```

slika 3-10 Vraćanje elemenata unutar roditeljske <div> oznake [22]

Za razliku od HTML-a, JSX svaku oznaku mora zatvoriti znakom /. Za primjer uzmimo HTML oznaku koja u JSX-u mora imati znak zatvaranja .

Atributi unutar JSX se uglavnom pišu camelCaseom što znači da će se, na primjer, stroke-width pisati isključivo strokeWidth. U slučaju nepoštivanja nekog od ovih pravila kompajler javlja grešku u kodu.

Iako korištenje JSX-a u Reactu nije obavezno, mnogi developeri se odlučuju na to rješenje s obzirom da ga mnogi smatraju kao pomoć u vizualnom i organizacijskom smislu.

- React Hooks

React Hooks je jedna od novijih elemenata Reacta koja je u programiranje uvela mnogo novih stvari. Hookovi developerima omogućuju korištenje različitih React značajki iz stvorenih komponenti. React osigurava već ugrađene Hookse, ali omogućava i stvaranje vlastitih kombiniranjem postojećih.

State Hooks osiguravaju komponentama pamćenje informacija koje korisnik unese. Na slici 3-11 je prikazan način implementacije useState hooka koji u varijablu indeks sprema vrijednost 0, a kasnije u kodu ta vrijednost se mijenja set funkcijom setIndex te se može postaviti na bilo koju drugu vrijednost. Osim useState hooka postoji i useReducer hook koji se koristi za složeniju logiku stanja.


```
function ImageGallery() {  
  const [index, setIndex] = useState(0);  
  // ...  
}
```

slika 3-11 Primjer korištenja useState hooka

Context Hooks omogućavaju prosljeđivanje informacija od udaljenih roditeljskih komponenti kao argumente. Za primjer uzmimo temu pozadine koju možemo slati iz glavne komponente u sve ostale komponente bez obzira na njihovu udaljenost u odnosu na glavnu komponentu.

Ref Hooks služe za spremanje podataka koji se ne koriste u renderiranju web stranice te samim time njihova promjena ne uzrokuje njeno ponovno renderiranje.

Effect Hooks se koriste za povezivanje i sinkronizaciju s unutarnjim sustavima aplikacije, uključujući animacije, DOM komponente i ostali ne-React kod. Koriste se kada je potrebno ponovno renderanje web stranice nakon određenog događaja koji korisnik napravi.

Osim ovih osnovnih postoje i drugi React hooksi koji su integrirani, ali također koji mogu biti i stvoreni od strane developera kombiniranjem postojećih. [23]

3.2.3. Sintaksa i varijable

Kao što je već napomenuto u prijašnjim podnaslovima, sintaksa koja se koristi u Reactu je JSX, dok se varijablama upravlja React Hooksovima. Obje ove stvari su u React dodane kako bi se developerima olakšalo pisanje i razumijevanje kodova.

- Varijable

Budući da se React koristi JavaScript jezikom za pisanje kodova, varijable koje postoje su const, let i var, koje imaju različite opsege unutar koda.

Var je stara varijabla koja se koristila prije ES6 standarda te ima funkcijski scope što znači da je vidljiva samo unutar funkcije u kojoj je definirana. Također može biti i redeklarirana i redefinirana unutar istog scopea što može dovesti do neželjenih grešaka.

Problemi var varijable riješeni su uvođenjem let i const varijabli koje su uvedene ES6 standardom. Obje navede imaju blok scope, vrijede unutar bloka omeđenog vitičastim zagradama {}. Kod let varijable vrijednost može biti promijenjena, ali ista varijabla ne može

biti redeklarirana u istom scopeu. Const varijabla je konstantna što znači da vrijednost ne može biti ponovno dodijeljena te se pri inicijalizaciji navedene varijable vrijednost odmah mora postaviti.

Za upravljanje stanjima varijabli koriste se React Hookovi koji korisniku omogućuju lakše korištenje samih varijabli. Imenovanje varijabli također ima određena pravila, na primjer varijable koje spremaju booleanske vrijednosti započinju s `is`, `has` ili `should`. U `useState` Hookovima praksa je da se za ažuriranje stanja varijable koristi `set` kao prefiks imenu varijable. (slika 3-12)

```
// Prefix state variables with **is**, **has**, or **should**

const ExampleComponent: React.FC = () => {
  const [isActive, setIsActive] = useState(false); // ✓
  const [hasError, setHasError] = useState(false); // ✓
  const [shouldRender, setShouldRender] = useState(true); // ✓

  // Avoid ✗
  const [active, setActive] = useState(false); // ✗
  const [err, setErr] = useState(false); // ✗
  const [render, setRender] = useState(true); // ✗
}
```

slika 3-12 Pravila pisanja varijabli [24]

- Pisanje komponenti

Imenovanje komponenti u Reactu također ima svoja pravila. Za pisanje naziva komponenti se koristi PascalCase (svako slovo nove riječi je veliko, počevši s prvom riječi).

```
// React component ✓
const TodoItem = () => {
  ...
}

// Avoid ✗
const todoItem = () => {
  ...
}
```

slika 3-13 Imenovanje komponenti [24]

Što su komponente manje to je developerima lakše pisanje i održavanje koda. Na slikama 3-14 i 3-15 je prikazano korištenje komponente unutar komponente, način njihovog zapisa, imenovanje te prijenos atributa iz jedne u drugu.

```

1  import React from "react";
2  import "./infoScreen.css";
3  import TextForInfo from "../textForInfo/textForInfo";
4
5  const InfoScreen = ({ visible, setIsInfoVisible, isHelp }) => {
6    if (!visible) return;
7
8    return (
9      <div className="screen-blur">
10        <div className="info-screen">
11          <button
12            className="exit-button"
13            onClick={() => {
14              setIsInfoVisible(false);
15            }}
16          >
17            X
18          </button>
19          <TextForInfo isHelp={isHelp} />
20        </div>
21      </div>
22    );
23  };
24
25  export default InfoScreen;
26

```

slika 3-14 Prikaz pozivanja komponente *TextForInfo* s atributom iz komponente *InfoScreen*

```

1  import React from "react";
2  import "../textForInfo.css";
3  import InfoBlock from "../infoBlock/infoBlock";
4  import HelpBlock from "../helpBlock/helpBlock";
5
6  const TextForInfo = ({ isHelp }) => {
7    if (isHelp) {
8      return (
9        <div className="text">
10          <HelpBlock />
11        </div>
12      );
13    } else {
14      return (
15        <div className="text">
16          <InfoBlock />
17        </div>
18      );
19    }
20  };
21
22  export default TextForInfo;
23

```

slika 3-15 Komponenta *TextForInfo* koja poziva manje komponente

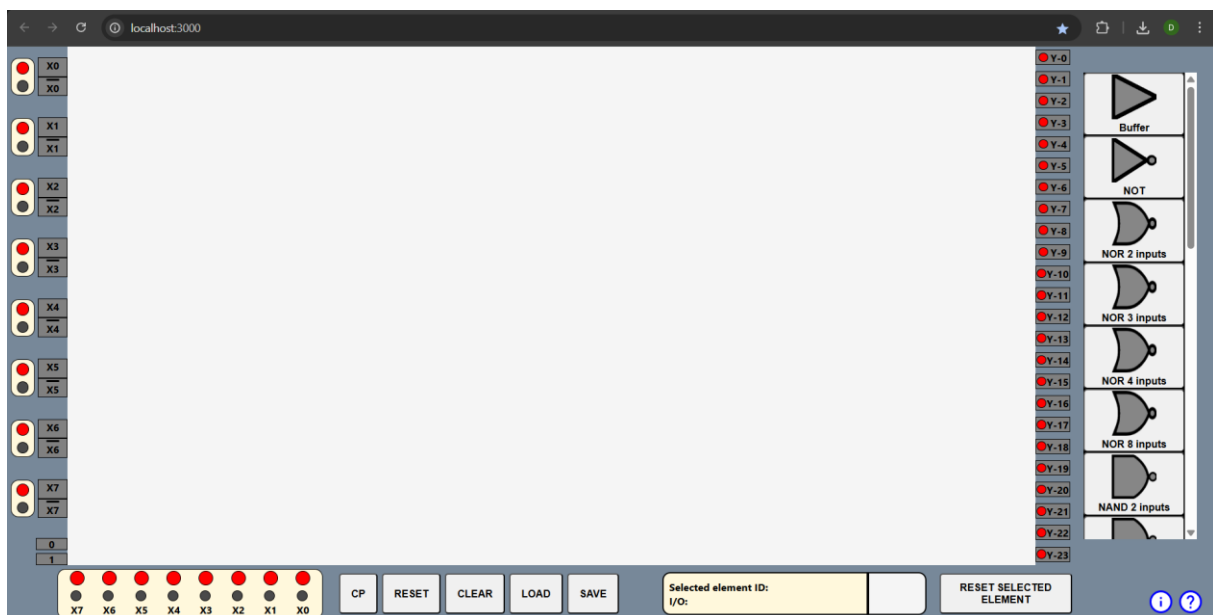
4. APLIKACIJA

Ovo poglavlje opisuje specifično zadatak ovog završnog rada, a to je „Simulacija digitalnih sklopova korištenjem React.js“. Radi jednostavnosti, u sljedećem će tekstu navedeni sustav biti adresiran sa „aplikacija“.

U ovom je dijelu samo ukratko opisana funkcionalnost aplikacije: web aplikacija koja omogućava dodavanje logičkih sklopova te generiranje njihovih izlaza. Korisnik može lokalno spremiti kreirani digitalni sklop te učitati jedan od predefiniranih ili prethodno spremljenih sklopova.

4.1. Grafičko sučelje

Grafičko sučelje ove web aplikacije je kreirano na način da korisnicima bude što jednostavnije i intuitivnije korištenje različitih značajki aplikacije (slika 4-1). Također općeniti izgled sučelja je uzet iz modela DELAB1 i razvojnog sustava STK200 koji se koristi na laboratorijskim vježbama na predmetu Digitalna i mikroprocesorska tehnika.

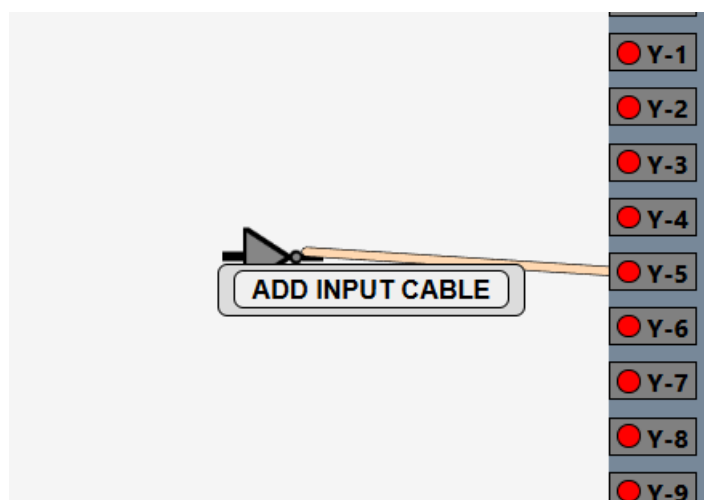


slika 4-1 Grafičko sučelje aplikacije

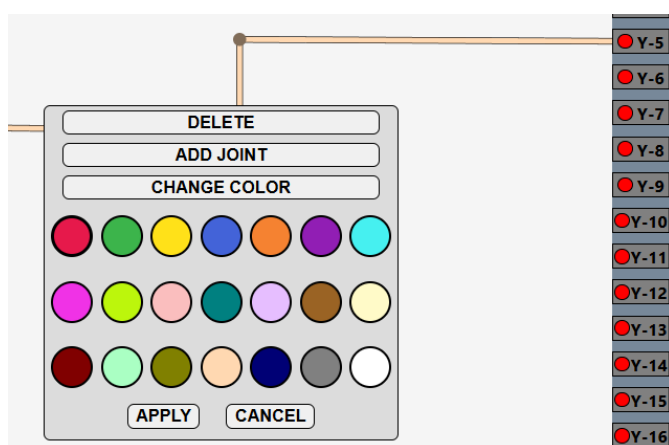
S lijeve strane se nalazi generator s osam varijabli od X0 do X7 koje sadrže i njihove negirane vrijednosti. Crvena svijetla kraj oznaka varijabli simboliziraju stanje varijable s crvenim za 0 te zelenim za 1, dok gumbi ispod svijetala omogućuju mijenjanje njihovih vrijednosti. Na dnu lijevog dijela se nalaze logički izlazi 1 i 0 kao konstante.

S desne strane je prvo prikazan dio s 24 indikatora, od Y0 do Y23, koji na isti princip prikazuju crveno svjetlo ukoliko spojeni sklop daje izlaz 0, inače se pali zeleno svjetlo. Na samom desnom rubu se nalazi izbornik za dodavanje elemenata. U izborniku se scrollbarom traže elementi, a klikom na pojedini element se taj element dodaje na središnje područje.

Središnje područje omogućava „drag and drop“ elemenata po cijelom području te dodavanje kablova za njihovo međusobno spajanje. Desnim klikom na ulaznu/izlaznu točku elementa, generatora i indikatora otvara se izbornik (slika 4-2) za dodavanje kabela. Desni klik izravno na element otvara drugi izbornik koji omogućuje brisanje samog elementa, a desni klik na kabel otvara treći izbornik preko kojeg se može mijenjati boja kabela, dodavati zglob kabela ili ga se brisati kao što je prikazano na slici 4-3. Hoveranjem preko elementa prikaže se njegov id koji je u obliku TipElementa-broj, npr. PriorityEncoder-0.

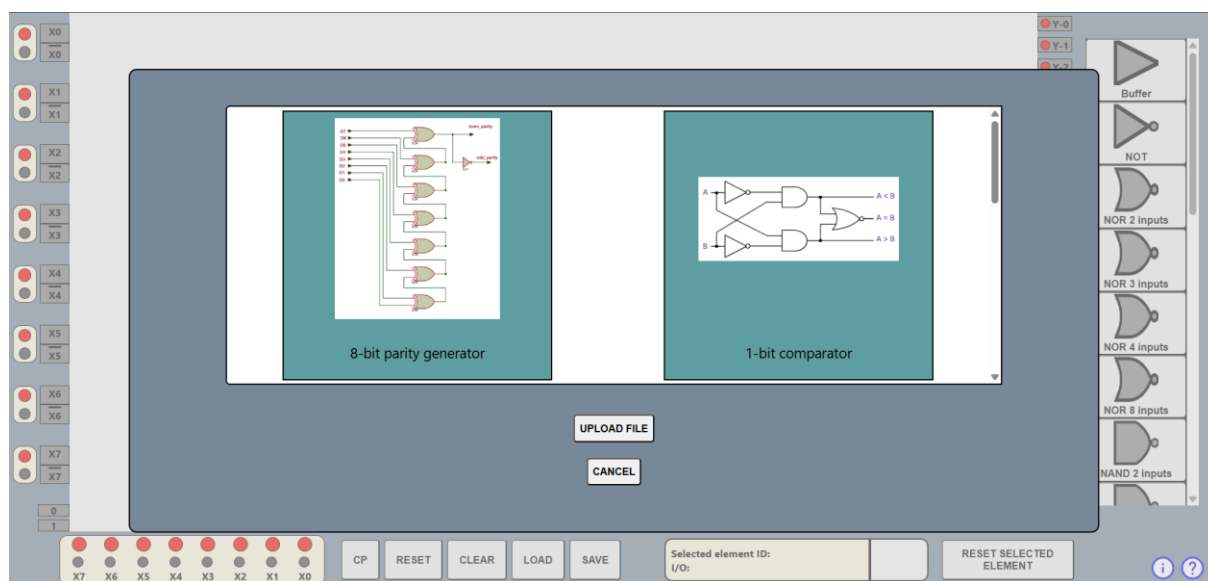


slika 4-2 Otvaranje izbornika za dodavanje kabela



slika 4-3 Izbornik kabela

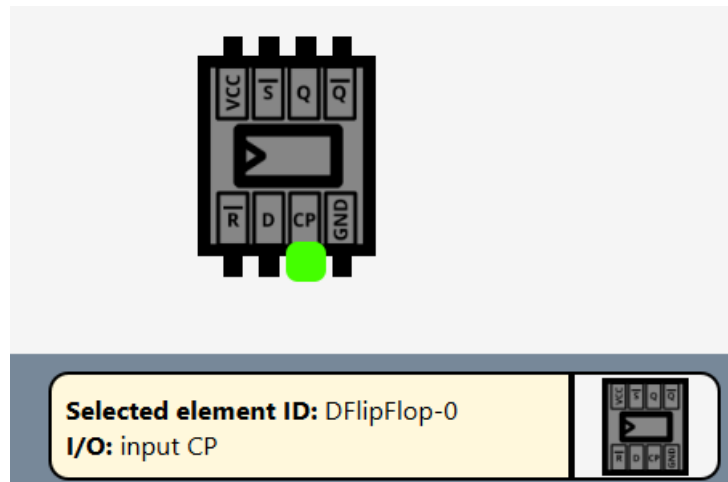
Donji dio ekrana je rezerviran za kontrolnu ploču. Lijevi dio ploče dozvoljava mijenjanje vrijednosti varijabli generatora, kao i tipkama pored samog generatora, ali ovo omogućuje korisniji vizualni prikaz (ako se broj 5 prikazuje binarno prikazano će biti 00000101). Pored generatora nalazi se tipka CP koja omogućava inkrementiranje generatora za binarno 1. RESET tipka sve varijable vraća na vrijednost 0, a CLEAR briše sve elemente i kabele sa središnjeg područja. LOAD otvara izbornik za dodavanje predefiniranih logičkih sklopova ili za učitavanje sklopa s računala kao što je prikazano na slici 4-4. SAVE omogućava spremanje trenutno složenog sklopa koji se kasnije može učitati LOAD tipkom.



slika 4-4 Otvoreni izbornik nakon pritiska tipke LOAD

Idući prikaz na kontrolnoj ploči označava koji ulaz/izlaz kojeg elementa je odabran za dodavanje kabela te je to vidljivo i indikatorom na središnjoj ploči. Na slici 4-5 je prikazan odabir ulaza CP na D bistabilu s idjem DFlipFlop-0. Odabirom idućeg mjesta za dodavanje kabela, kabel se stvara, a odabrani element se postavlja na prazno. Isti učinak postavljanja elementa na prazno ima i tipka RESET SELECTED ELEMENT.

Tipke na samom kraju kontrolne ploče otvaraju informacije o developeru projekta i projektu te pomoć za korištenje same aplikacije koja objašnjava kako se spajaju pojedini elementi te tipke na kontrolnoj ploči.



slika 4-5 Prikaz odabira ulaza na koji se dodaje kabel

4.2. Struktura elemenata

Svaki logički element korišten u ovoj aplikaciji je dizajniran na jednak način. Pojedini element sadrži id, sliku, oznake za dodavanje kablova, te logiku za računanje izlaza iz danih ulaza. Njegova kontrola je omogućena „drag and drop“ mehanizmom te izbornicima za dodavanje kablova, brisanje elementa te prikaz idja hoveranjem preko elementa.

U samoj komponenti za svaki ulaz/izlaz elementa postoji funkcija koja rukuje klikom na ulaz/izlaz elementa, npr.

```
const handleNOT1InputClick = (e) => {
  const inputNumber = 1;
  const newElement = {
    id: element.id + "-input1",
    value: element.value,
  };
  handleElementClick(newElement, e, 0, inputNumber);
};
```

koja na id elementa dodaje nastavak za broj ulaza te postavlja vrijednost elementa. To se sve skupa šalje u funkciju koja rukuje klikom na cjelokupni element. Preko te funkcije iz poslanih atributa se računa pozicija na koju se kabel treba dodati, postavlja se vrijednost odabranog elementa na sami element te se zatvaraju izbornici ukoliko ima ijedan otvoren.

Osim funkcija za rukovanje klikom na element u sami return komponente se za svaki element dodaje dio koji omogućava „drag and drop“:

```
<Draggable
```

```

axis="both"
handle=".handle"
defaultPosition={element.position}
position={null}
scale={1}
onDrag={handleDrag}
bounds=".draggable-area"
nodeRef={draggableRef}
>

```

Prikazani kod omogućava micanje elementa na obje osi (x i y) držanjem lijevog klika i micanjem miša. Postavljena je početna pozicija elementa i granice unutar kojih je dopušteno micanje elementa, u ovom slučaju središnje područje „draggable-area“. nodeRef se koristi kao referenca na sami element.

4.3. Algoritam za izračun izlazne vrijednosti

Dok same komponente elemenata sadrže vizualne aspekte za njihov prikaz, algoritmi za izračun izlaznih vrijednosti spremljeni su u čiste JavaScript datoteke.

Glavni dio koda koji omogućava izračun izlaznih vrijednosti postignut je rekurzivnom funkcijom „findPath“ koja za svaki indikator Y traži put preko svih kabela i elemenata povezanih na taj izlaz do generatora te sprema sve elemente koji su se našli na putu između indikatora i generatora.

```

const findPath = (cables, path, yId) => {
  const idToCheck = yId.split("-")[0] + "-" + yId.split("-")[1];
  cables.forEach((cable) => {
    const cableElementIdToCheck =
      cable.element2.id.split("-")[0] + "-" + cable.element2.id.split("-")[1];
    if (cableElementIdToCheck === idToCheck) {
      const newElement = {
        ...cable.element2,
        calculatedValue: undefined,
      };
      path.push(newElement);
      if (cable.element1.id.split("-")[0] !== "X") {
        findPath(cables, path, cable.element1.id);
      } else {
        let newElement;
        if (cable.element1.id.split("-")[2] === "NOTX") {
          newElement = {
            ...cable.element1,
            calculatedValue: cable.element1.value === "1" ? "0" : "1",
          };
        }
      }
    }
  });
};

```



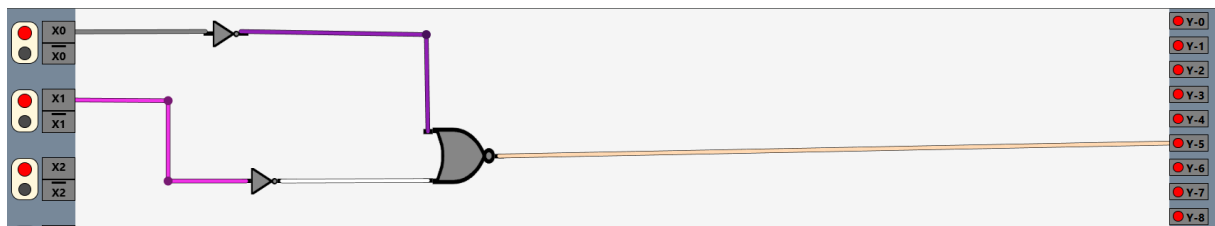
```

    };
  } else {
    newElement = {
      ...cable.element1,
      calculatedValue: cable.element1.value === "1" ? "1" : "0",
    };
  }
  path.push(newElement);
}
}
});
};

```

Funkcija radi na principu da za svaki kabel, koji sadrži idjeve od svog početnog i krajnjeg elementa, traži početni element, to se ponavlja dok se svaki element kojem je id različit od X, id generatora, nije spremio u varijablu „path“. Na kraju se i vrijednosti samog generatora spremaju u istu varijablu koja sada sadrži sve elemente potrebne za izračun konačne vrijednosti logičkog sklopa.

Dodatno ću objasniti princip rada funkcije „findPath“ na primjeru sa slike 4-6.



slika 4-6 Jednostavan logički sklop

U funkciju „findPath“ se ulazi svim kabelima, varijablom path koja je prazna te indikatorom Y-5. Na početku se traži kabel kojem je id izlaznog elementa jednak idju indikatora. Ukoliko se ne nađe nijedan takav kabel indikator nije spojen na nikoji element, ali u slučaju Y-5 se kabel pronalazi. Stvara se prvi član varijable path koji ima svoj id Y-5 te nedefiniranu vrijednost. Nakon toga se provjerava ulazni element kabla, ako mu id nije ni X ni NOTX, ponovno se pokreće funkcija „findPath“ koja se sad šalje ponovno sa svim kablovima, pathom koji sad ima prvi element Y-5 te idjem novog elementa u ovom slučaju NOR2Inputs-0. Sada se ponavlja prethodni postupak koji na isti način pronalazi kabele kojima je izlazni element NOR2Inputs-0 te uzima njihove ulazne elemente koji u ovom slučaju ponovno nisu ni X ni NOTX te se oni dodaju u varijablu path. Nakon prve iteracije sprema se NOR2Inputs-0-input1, pa NOT-0-input1 te se na kraju dolazi do elementa X-0 koji, budući da mu je id X, se sprema u varijablu path te se izlazi iz rekurzivne funkcije. Ista stvar se događa na drugoj grani gdje se redom

spremaju NOR2Inputs-0-input2, NOT-1-input1 te X-1. Na kraju svih iteracija za indikator Y-5 path varijabla sadrži sve članove redom: Y-5, NOR2Inputs-0-input1, NOT-0-input1, X-0, NOR2Inputs-0-input2, NOT-1-input1 te X-1 sa njihovim vrijednostima, nedefinirane za sve osim X-0 i X-1.

Tako zadan put od ulaza do izlaza se šalje u drugu funkciju:

```
const calculateOutput = (path, cables) => {
  const indexOfY =
    path.findIndex((element) => element.id.split("-")[0] === "Y") + 1;
  let counter = 0;
  const pathToCheck = path.slice(0, indexOfY);
  if (pathToCheck.length > 1) {
    while (counter < indexOfY) {
      if (pathToCheck[counter].calculatedValue === undefined) {
        const valueToAssign = chooseElementsForCalculation(
          counter,
          pathToCheck,
          cables
        );
        pathToCheck[counter].calculatedValue = valueToAssign;
      }
      counter++;
    }

    return pathToCheck[counter - 1].calculatedValue;
  } else {
    return "0";
  }
};
```

koja za elemente iz path varijable, kojima je vrijednost nedefinirana, računa te vrijednosti u funkciji „chooseElementForCalculation“ koja na temelju elementa kojem se traži vrijednost te prethodnika tog elementa računa izlaznu vrijednost.

Ponovno uzmimo isti primjer za koji smo path već izračunali. Princip na koji se računa izlaz za određeni element je slijedeći: pronađi sve elemente u pathu koji imaju isti id (ne uzimajući u obzir broj ulaza) te za svaki taj element nađimo prethodnika preko kojeg ćemo izračunati izlaz. U primjeru redom krećemo s X-1 koji je već definiran pa se preskače, nakon njega slijedi NOT-1 koji se više u nizu ne ponavlja pa se njegova vrijednost računa preko samo njegovog prethodnika X-1 kojemu je sad vrijednost 0 pa je vrijednost NOT-1 1. Idući element je NOR2Inputs-0 koji se pojavljuje na dva mjesta te su njegovi prethodnici NOT-1 i NOT-0, ali budući da NOT-0 još nije izračunat preskačemo tu iteraciju. Slijedi X-0 i NOT-0 kojem se

računa vrijednost preko poznatog X-0 te NOT-0 dobiva vrijednost 1. Niz se nastavlja s ponovno NOR2Inputs-0 koji se ponavlja dva puta, ali ovaj put su mu oba prethodnika s određenom vrijednosti 1 i za NOT-0 i za NOT-1. Preko njih se računa vrijednost NOR2Inputs-0 koja se postavlja na 0. Nakon toga slijedi Y-5 kojem se vrijednost daje preko njegovog prethodnika tj. vrijednost mu je 0 što je na slici 4-6 i vidljivo. Za složenije primjere i elemente, računanje izlaza radi na isti način.

4.4. Predefinirani i učitani digitalni krugovi

Još jedna od korisnih značajki ove aplikacije je učitavanje predefiniranih krugova, spremanje kreiranih te učitavanje tih istih.

Spremanje kreiranih krugova se vrši u funkciji:

```
const saveFile = async (allElements, allCables) => {
  getAllElementsFromDisplay(allElements, allCables);

  const data = {
    elements: savedDisplayElements,
    cables: savedDisplayCables,
    timestamp: new Date().toISOString(),
  };

  const encryptedData = CryptoJS.AES.encrypt(
    JSON.stringify(data),
    secretKey
  ).toString();

  try {
    const fileHandle = await window.showSaveFilePicker({
      suggestedName: "logical-circuit.clc",
      types: [
        {
          description: "Custom Layout Config",
          accept: { "application/clc": [".clc"] },
        },
      ],
    });

    const writable = await fileHandle.createWritable();
    await writable.write(encryptedData);
    await writable.close();

    alert("File saved successfully!");
  }
}
```

```
    } catch (error) {  
        console.error("Error saving file:", error);  
    }  
};
```

Na početku funkcije se traže svi elementi stavljeni na središnji ekran, kablovi i sami elementi, koji se spremaju u objekt dana koji se enkriptira po sigurnosnom ključu. Korak enkripcije nije nužan u ovom slučaju s obzirom da se u datoteku spremaju samo elementi i kablovi. Try-catch blok provjerava uspješnost spremanja datoteke koja se na računalo sprema s nastavkom .clc.

Na istom principu radi i učitavanje datoteka. Uzmemo se spremljeni podaci, enkriptiraju se u novi objekt te ako je datoteka korumpirana ili sadrži neku grešku try-catch blok nas upozorava i prekida radnju. Ako je enkripcija bila uspješna elementi i kablovi se iz objekta postavljaju u svoja polja, cables koji sadrži sve kablove te buffer, NOT, NOR2Inputs... koja sadržavaju sve elemente određenog tipa. Aplikacija iz tako spremljenih polja učitava kablove i elemente te ih postavlja po njihovim lokacijama na središnji dio.

Predefinirani logički sklopovi su neki od češće korištenih korisnih sklopova: 1 bitni komparator, 3 bitni komparator, 2 bitno zbrajalo, poluzbrajalo te 8 bitni generator pariteta.

5. ZAKLJUČAK

Otkad tehnologija u novije vrijeme sve brže napreduje, uporaba i znanje o logičkim sklopovima sve je potrebnije, budući da se svi elektronički elementi proizvode uporabom samih logičkih sklopova. Ovaj rad za cilj ima dodatno unaprijediti znanja i načine rješavanja poteškoća u realizaciji kombiniranja logičkih sklopova.

Sustavi za simulaciju rada logičkih sklopova su moćni alati koji se često koriste u obrazovnom sektoru kako bi se studentima što više približio rad samih sklopova, a da se izbjegava potreba za njihovim fizičkim korištenjem. Osim u obrazovnom sektoru mnogi inženjeri koriste ovakve simulacije kako bi provjerili uspješnost i korisnost kombiniranja različitih elemenata.

Uporabe ovakvih tehnologija mogu pomoći studentima, ali i samim obrazovnim ustanovama jer se štedi i vrijeme i resursi koji bi bili potrebni za fizičko posjedovanje te povezivanje elemenata.

U zaključku, ovakvi sustavi donose brojne prednosti, profesorima, studentima, inženjerima za što bolje izvršavanje zadataka u kombiniranju sklopova. Njihova primjena može realizirati učinkovitijem procesu učenja, testiranja i implementiranja samih logičkih sklopova te time poboljšati radno iskustvo svim uključenim stranama.

LITERATURA

- [1] Hrvatska Enciklopedija, logički sklopovi, s interneta: <https://enciklopedija.hr/clanak/logicki-sklopovi>, 28.5.2025.
- [2] Logisim, Features, s interneta: <https://www.cburch.com/logisim/>, 26.5.2025.
- [3] hneemann, Digital, s interneta: <https://github.com/hneemann/Digital>, 26.5.2025.
- [4] logic.ly, Teach logic gates + digital circuits effectively — with Logically, s interneta: <https://logic.ly/>, 26.5.2025.
- [5] Geeks for Geeks, Multiplexers in Digital Logic, s interneta: <https://www.geeksforgeeks.org/multiplexers-in-digital-logic/>, 28.5.2025
- [6] Geeks for Geeks, What is Demultiplexer(DEMUX)?, s interneta: <https://www.geeksforgeeks.org/what-is-demultiplexerdemux/>, 28.5.2025.
- [7] Geeks for Geeks, D Flip Flop, s interneta: <https://www.geeksforgeeks.org/d-flip-flop/>, 28.5.2025.
- [8] Geeks for Geeks, What is JK Flip-Flop, s interneta: <https://www.geeksforgeeks.org/what-is-jk-flip-flop/>, 28.5.2025.
- [9] Electronics Tutorial, Priority encoder, s interneta: https://www.electronics-tutorials.ws/combination/comb_4.html, 28.5.2025.
- [10] Microsoft, Code in any language, s interneta: <https://code.visualstudio.com/>, 21.5.2025.
- [11] Microsoft, Code anywhere, s interneta: <https://code.visualstudio.com/>, 21.5.2025.
- [12] Visual Studio Magazine, Stack Overflow Dev Survey: VS Code, Visual Studio and .NET Shine, s interneta: <https://visualstudiomagazine.com/articles/2024/07/26/so-dev-survey.aspx>, 21.5.2025.
- [13] Tower, VS Code — The Story and Technology Behind One of the World's Most Popular Desktop Apps for Developers, s interneta: <https://www.git-tower.com/blog/developing-for-the-desktop-vscode>, 21.5.2025.
- [14] GitHub, GitHub and Visual Studio Code, s interneta: <https://vscode.github.com/>, 21.5.2025.
- [15] Microsoft, Extensions for Visual Studio Code, s interneta: <https://marketplace.visualstudio.com/vscode>, 21.5.2025.

- [16] React, The library for web and native user interfaces, s interneta: <https://react.dev/>, 22.5.2025.
- [17] React, Introducing JSX, s interneta: <https://legacy.reactjs.org/docs/introducing-jsx.html>, 22.5.2025.
- [18] Rising Stack, The History of React.js on a Timeline, s interneta: <https://blog.risingstack.com/the-history-of-react-js-on-a-timeline/>, 22.5.2025.
- [19] DhiWise, Building High-Performance Applications with React Fiber, s interneta: <https://www.dhiwise.com/post/building-high-performance-applications-with-react-fiber>, 22.5.2025.
- [20] W3schools, CSS Syntax, s interneta: https://www.w3schools.com/css/css_syntax.ASP, 28.5.2025.
- [21] React, Virtual DOM and Internals, s interneta: <https://legacy.reactjs.org/docs/faq-internals.html>, 22.5.2025.
- [22] React, Writing Markup with JSX, s interneta: <https://react.dev/learn/writing-markup-with-jsx>, 22.5.2025.
- [23] React, React Hooks, s interneta: <https://react.dev/reference/react/hooks>, 22.5.2025.
- [24] Dev, React JS - Naming convention, s interneta: <https://dev.to/kristiyanvelkov/react-js-naming-convention-lcg>, 26.5.2025.

POPIS OZNAKA I KRATICA

MUX	multiplekser
DEMUX	demultiplekser
D	delay
JK	Jack Kilby
VS	Visual Studio
OS	operacijski sustav
CSS	Cascading Style Sheets
DOM	Document Object Model
VDOM	virtualni DOM
JSX	JavaScript Extensible Markup Language
HTML	Hypertext Markup Language

SAŽETAK

Ovaj rad predstavlja razvijenu web aplikaciju za simulaciju logičkih sklopova pomoću React.js-a. Aplikacija pruža uvid u ponašanje različitih digitalnih sklopova korištenjem grafičkih komponenti korisničkog sučelja. Rad definira karakteristične pojmove koji su važni za razumijevanje funkcioniranja logičkih sklopova i daje pregled postojećih rješenja vezanih uz simulaciju osnovnih digitalnih sklopova. Predstavljena je tehnologija korištena za izradu aplikacije, s naglaskom na karakteristične značajke koje su važne za performanse i mogućnost stvaranja rješenja. Opisane su funkcionalnosti aplikacije i način na koji su dostupne putem grafičkih komponenti sučelja. Istaknute su posebnosti aplikacije u smislu mogućnosti spremanja i učitavanja datoteka unaprijed izgrađenih logičkih sklopova te algoritam za izračunavanje izlaznih vrijednosti iz logičkih sklopova na temelju zadanih ulaznih vrijednosti. Definirana funkcionalnost aplikacije i način na koji su prikazani dizajn i testiranje potvrđeni su pomoću karakterističnih primjera.

KLJUČNE RIJEČI

logički sklopovi, web aplikacija, Visual Studio Code, React.js

SIMULATION OF DIGITAL CIRCUITS USING React.js

SUMMARY

This thesis presents a developed web application for simulating logic circuits using React.js. The application provides insight into the behavior of various digital circuits using graphical user interface components. The paper defines characteristic terms that are important for understanding the functioning of logic circuits and provides an overview of existing solutions related to the simulation of basic digital circuits. The technology used to create the application was presented, with an emphasis on characteristic features that are important for performance and the possibility of creating solutions. The functionalities of the application and the way in which they are available through the graphical components of the interface are described. The particularities of the application in terms of the possibility to save and load files of pre-built logic circuits and the algorithm for calculating output values from logic circuits based on given input values are highlighted. The defined functionality of the application and the way in which design and testing are presented are confirmed using typical examples.

KEYWORDS

Logical Circuits, Web Application, Visual Studio Code, React.js